



Einführungsaufgaben 10 & 11

Version 07-08-2026

von

Yashar Busch
Oliver Gröne
Amelie Meyer
Daniel Pasternak

Hannover, 12. August 2026

Inhaltsverzeichnis

1	Aufgabe 10 - Konzept des Schedulers	1
1.1	Scheduling-Periode	1
1.1.1	Einflussparameter	1
1.2	Tasks	1
1.2.1	Scheduler	2
1.2.2	Uhr-Task	3
1.2.3	Reaktions-Task	3
1.2.4	Berechnungs-Task	4
1.3	Initialisierung	4
1.3.1	Stackpointer	4
1.3.2	Timer 0	5
1.3.3	Timer 2	5
1.3.4	Interrupts	5
1.3.5	Scheduler	5
1.3.6	Uhr-Task	5
1.3.7	Reaktions-Task	5
1.3.8	Berechnungs-Task	6
2	Aufgabe 11 - Monitoring	7
2.1	Unterschiedliche Ansätze des Monitorings	7
2.2	Konzept eines Software-Monitoring für 8051	8
2.3	Laufzeitsimulation	8

1 Aufgabe 10 - Konzept des Schedulers

Im Konzept des Schedulers gehen wir auf die Tasks, Scheduling-Periode, die Stacks und die Spezialfunktionsregister (SFR) ein.

1.1 Scheduling-Periode

Die Scheduling-Periode beträgt 2,5 ms. Somit ist die Echtzeitanforderung von 10 ms für den Reaktions-Task gegeben, da jeder Task maximal 2,5 ms läuft, was bei 4 Tasks summiert 10 ms sind.

Wir verwenden Timer 0 im Modus 1 (16-Bit Timer). Timer 0 muss bei jedem Aufruf eines Tasks neu gesetzt werden, da in dem Modus der Timer nicht automatisch Nachladen kann und wir ggf. auch vor einem Überlauf in einem alten Task in einen neuen Task wechseln. Timer 0 wird auf den Wert 0xF36C bzw. 63.036 gesetzt.

1.1.1 Einflussparameter

Die Aufgabenstellung verlangte über verschiedene Einflussparameter auf die Scheduling-Periode zu diskutieren.

Hitze Durch Hitze kann es zu einer Verlangsamung des Systems bzw. des Timers kommen. Dies kann zu einer Verlängerung der Scheduling-Periode führen.

Interrupts Sollten im Zuge einer Umstellung der Tasks andere Interrupts eingeschaltet werden, so kann es dazu kommen, dass der Timer 0 Interrupt erst verspätet abgearbeitet wird. Dies kann zu einer Verlängerung der Scheduling-Periode führen.

1.2 Tasks

Jeder Task (einschließlich des Schedulers) hat eine eigene Registerbank zugewiesen (siehe bei den Taskbeschreibungen die Eckdaten). In der Regis-

terbank des Schedulers bekommt jeder Task außerdem ein eigenes Register, in welchem wir die Stackpointer (SP) der jeweiligen Tasks speichern und auch auf diesen laden. Im Folgenden schauen wir uns die Tasks im einzelnen genauer an.

1.2.1 Scheduler

Der Scheduler kümmert sich um den Wechsel der Tasks bzw. des Kontexts und besitzt dazu seinen eigenen Kontext. Er sitzt in der Interrupt-Routine von Timer 0, unserem Scheduling-Periode-Timer.

Beim Aufruf des Schedulers wird automatisch, da es eine Interrupt Routine ist, der Programcounter (PC) auf den Stack des alten Tasks geschrieben. Innerhalb des Schedulers werden dann die SFR ebenfalls auf den Stack geschrieben (Reihenfolge: Akkumulator, B-Register, Programmstatuswort (PSW)-Register, Datapointer (DPTR)-Lowbyte (DPL) und DPTR-Highbyte (DPH)), die Registerbank wird zu der des Schedulers gewechselt und der SP des alten Tasks wird in das ihm zugewiesene Register gespeichert.

Im Folgenden wählt der Scheduler den nächsten Task, wechselt den Stack zum neuen Task, liest die SFR vom Stack und beendet die Interrupt-Routine, sodass der PC einfach wieder vom Stack gelesen wird und der Task da weitermachen kann, wo er aufgehört hat.

Zusammengefasst noch einmal der Ablauf des Schedulers:

1. Pushen der SFR auf den Stack des alten Tasks
2. Wechseln der Registerbank zu der dem Scheduler zugehörigen
3. Laden des Stacks des Schedulers
4. Laden der SFR aus dem Scheduler Stack
5. Nächsten Task auswählen
6. Pushen der SFR auf den Scheduler Stack
7. Laden des Stacks des neuen Tasks

Eckdaten des Schedulers:

Stackpointer (Init): 0x80

Registerbank: 0x00

Register für SP: R4 bzw. 0x04

1.2.2 Uhr-Task

Den Uhr-Task haben wir bereits in einer vorherigen Aufgabe erstellt, weshalb eine genau Erklärung der Funktionsweise hier ausbleibt und nur die Änderungen erklärt werden.

In der Vorlesung wurde sich darauf geeinigt, dass der Uhr-Task statt wie vorher mit dem Timer 0 und per Interrupt maximal alle 250 μ s schaltet, nun mit dem Timer 2 und alle 50 ms schalten soll. Durch die Aufgabenstellung war außerdem vorausgesetzt, dass der Scheduler „als einziger Task mit Hilfe eines Interrupts gesteuert wird.“

Durch diese Voraussetzungen wurde der Uhr-Task so abgeändert, dass wir ...

- ... keinen Interrupt verwenden, sondern bei jedem Aufruf des Uhr-Tasks das Interrupt-Bit abfragen,
- bei gesetztem Interrupt-Bit die Uhr hochzählen und das Interrupt-Bit zurücksetzen,
- bei nicht gesetztem Interrupt-Bit wird direkt an den Scheduler abgegeben und
- statt der vorherig maximal 250 μ s jetzt 50 ms hochzählen.

Eckdaten des Uhr-Tasks:

Stackpointer (Init): 0xA0

Registerbank: 0x01

Register für SP: R5 bzw. 0x05

1.2.3 Reaktions-Task

Der Reaktions-Task „berechnet, ob der binäre 8-bit Wert an einem festzulegenden Port größer als 200 oder kleiner als 100 ist. Diese Information wird durch zwei Bits eines anderen festzulegenden Ports mindestens alle 10 ms aktualisiert.“ Für die Ports haben wir uns als Eingabe (8-Bit) für Port 1 entschieden. Als Ausgabe benutzen wir P3.2 (XH) und P3.3 (XL). Die Echtzeitanforderungen erfüllen wir wie in Abschnitt 1.1 erklärt. Außerdem haben wir uns dazu entschieden, bei Aufruf des Reaktions-Task den Wert des vorherigen Aufrufs gegen den Wert des aktuellen Aufruf zu vergleichen und bei Gleichheit direkt an den Scheduler abzugeben, um so CPU Zeit zu sparen. Dafür wird der anliegende Wert in R0 zwischengespeichert.

Eckdaten des Reaktions-Task:

Stackpointer (Init): 0xC0

Registerbank: 0x10

Register für SP: R6 bzw. 0x06

1.2.4 Berechnungs-Task

Im Berechnungs-Task „werden alle Speicherzellen des externen RAMs entsprechend der Größe sortiert.“ Der Einfachheit halber haben wir uns hier für einen Bubble-Sort entschieden. Wir vergleichen also immer die aktuelle Zelle mit der nächsten Zelle im externen RAM und tauschen, sofern die aktuelle Zelle größer ist. Eckdaten des Berechnungs-Tasks:

Stackpointer (Init): 0xE0

Registerbank: 0x11

Register für SP: R7 bzw. 0x07

1.3 Initialisierung

In der Initialisierung müssen verschiedene Werte gesetzt werden. Am Ende muss das Interrupt-Bit für Timer 0 gesetzt werden, um den Scheduler aufzurufen.

Die Initialisierung der Stacks ist dabei besonders. Hier ist auf eine gewisse Struktur zu achten, um später einen reibungslosen Ablauf im Scheduler zu gewährleisten. Die Struktur sieht wie folgt aus:

1. PC-Lowbyte
2. PC-Highbyte
3. Akkumulator
4. B-Register
5. PSW-Register
6. DPL-Register
7. DPH-Register

Die PC-Werte werden dynamisch durch die Label der Unterprogramme gesetzt. Das PSW-Register wird so initialisiert, dass es direkt auf die dem Task zugewiesene Registerbank zeigt. Alle anderen Register werden mit 0x00 initialisiert.

Der Ablauf ist im Folgenden beschrieben:

1.3.1 Stackpointer

Zuallererst wird der SP auf den Stack des Schedulers gesetzt. Unser SP für den Scheduler beginnt bei 0x80 und wird daher auf 0x7F gesetzt.

1.3.2 Timer 0

Timer 0 muss auf den Modus 1 (16-Bit) geschaltet werden. Er wird mit 0xF36C geladen und gestartet.

1.3.3 Timer 2

Timer 2 wird im Nachlademodus betrieben. Es müssen Timer 2 und die Nachladeregister des Timer 2 mit 0x3CB0 geladen werden. Der Timer wird gestartet.

1.3.4 Interrupts

Es wird der Interrupt für Timer 0 eingeschaltet. Es wird die globale Interruptfreigabe gesetzt.

1.3.5 Scheduler

Es wird in A 0x04 geschrieben.

1.3.6 Uhr-Task

Der SP des Uhr-Task beginnt bei 0xA0, wir setzen also SP als Nächstes auf 0x9F. Es werden die folgenden Werte auf den Stack geschrieben:

- Lowbyte Programmlabel
- Highbyte Programmlabel
- Akkumulator (0x00)
- B-Register (0x00)
- PSW-Register (0x08)
- DPL-Register (0x00)
- DPH-Register (0x00)

Der SP (nun 0xA6) wird im zugehörigem Register (R5) gespeichert.

1.3.7 Reaktions-Task

Der SP des Reaktions-Task beginnt bei 0xC0, wir setzen also SP als Nächstes auf 0xBF. Es werden die folgenden Werte auf den Stack geschrieben:

- Lowbyte Programmlabel
- Highbyte Programmlabel

- Akkumulator (0x00)
- B-Register (0x00)
- PSW-Register (0x10)
- DPL-Register (0x00)
- DPH-Register (0x00)

Der SP (nun 0xC6) wird im zugehörigem Register (R6) gespeichert.

1.3.8 Berechnungs-Task

Der SP des Berechnungs-Task beginnt bei 0xE0, wir setzen also SP als Nächstes auf 0xDF. Es werden die folgenden Werte auf den Stack geschrieben:

- Lowbyte Programmlabel
- Highbyte Programmlabel
- Akkumulator (0x00)
- B-Register (0x00)
- PSW-Register (0x18)
- DPL-Register (0x00)
- DPH-Register (0x00)

Der SP (nun 0xE6) wird im zugehörigem Register (R7) gespeichert.

2 Aufgabe 11 - Monitoring

2.1 Unterschiedliche Ansätze des Monitorings

Monitoring Ansatz	Vorteile	Nachteile
Software-Monitoring	• Einfach zu implementieren • Keine zusätzliche Hardware erforderlich	• Zugriff auf Kernel-Ebene benötigt • Höherer Zeitbedarf und Ressourcennutzung • Möglicherweise veraltete Daten • (mögliche) Beeinträchtigung des Echtzeit-Verhaltens
Hybrid-Monitoring	• Geringerer CPU-Overhead • Gute Balance zwischen Kosten/Genauigkeit	• Zusätzliche Hardware benötigt • Anpassungen im Quellcode notwendig
Hardware-Monitoring	• Unabhängig, da keine Systemressourcen benötigt werden	• Zusätzliche Hardware benötigt • Spezielles Werkzeug notwendig aufgrund der Größe

Das Software-Monitoring ist zwar einfach zu implementieren, jedoch verliert es in Sachen Performance und Genauigkeit, da die Daten über mehrere Ebenen durchgereicht werden und zudem eine Umwandlung von analog in digital geschieht, wodurch es zu Ungenauigkeiten kommt.

Beim Hybrid-Monitoring ist die Balance zwischen Leistung/Effizienz. Die eigentliche Messung geschieht über Hardware-Sensorik, jedoch muss der Quellcode angepasst werden, um diese Daten auslesen zu können. Daher bietet sich dies als Kompromiss an, falls Leistung „über“ ist.

Das Hardware-Monitoring ist am schnellsten, da keine zusätzlichen Ebenen benötigt werden, um die Daten auszulesen. Zudem ist die Genauigkeit ebenfalls höher, da die analogen Signale nicht in digitale umgewandelt werden müssen.

2.2 Konzept eines Software-Monitoring für 8051

eine Stunde Unsere Idee ist im internen RAM je Task vier Byte zu reservieren. Timer 1 zählt dann zur Laufzeit des jeweiligen Tasks mit. Wenn der Task gewechselt wird, wird die gelaufene Zeit aufaddiert und die Zellen gewechselt. Damit können wir einen unsigned integer mit 32-Bit speichern, dessen Wert in ms ca. einer Stunde und zwölf Minuten entspricht.

sechs Monate Das Vorgehen entspricht dem vorherigen. Dafür benötigen wir ca. 5,5 Byte. Diese runden wir auf sechs Byte auf. Mit dieser Anzahl wären wir sogar in der Lage 108 Monate zu speichern.

2.3 Laufzeitsimulation

Iteration	Init	Scheduler	C-Task	Scheduler	R-Task	Scheduler	Uhr
1.	103 μ s	64 μ s	2499 μ s	63 μ s	24 μ s	63 μ s	11 μ s
2.	-	69 μ s	2493 μ s	63 μ s	9 μ s	63 μ s	7 μ s
3.	-	69 μ s	2494 μ s	63 μ s	9 μ s	63 μ s	7 μ s
4.	-	69 μ s	2493 μ s	63 μ s	20 μ s	63 μ s	7 μ s
5.	-	69 μ s	2494 μ s	63 μ s	9 μ s	63 μ s	7 μ s
6.	-	69 μ s	2493 μ s	63 μ s	24 μ s	63 μ s	7 μ s
...	...	...	...	...	...	...	...
19.	-	69 μ s	2493 μ s	63 μ s	9 μ s	63 μ s	10 μ s
...	...	...	...	...	...	...	...
370.	-	69 μ s	2493 μ s	63 μ s	9 μ s	63 μ s	12 μ s

Abbildung 2.1: Laufzeiten der Tasks mit Extrapolation

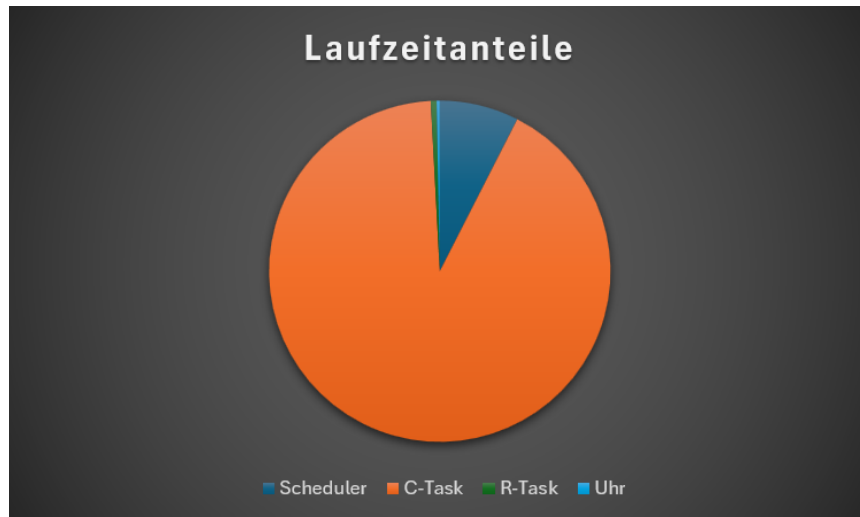


Abbildung 2.2: Laufzeitanteile

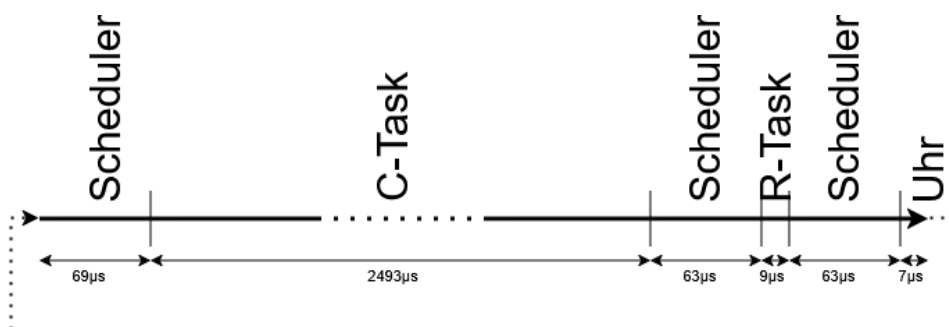


Abbildung 2.3: Zeitverlauf wiederkehrend